

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – PROMPT ENGINEERING CHALLENGE

Course Code:	CSA6501	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO2 – Precision (BL2, BL3)	Assessment:	Assessment Tool 1 – Transformer Mechanics Worksheet
Weightage:	40% of CIA	Total Marks:	2 * 50 = 100 (2 Questions)
CO1 Statement:	<i>pply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems (BL2, BL3)</i>		
Student Name:	A.Sai Rohit	Reg. No.:	192472144
Section / Batch:	CSE-AI	Date:	

Instructions to Students

- Answer 2 questionss.Each questions carries 50 marks..Total100 Marks.
- 1. Design appropriate prompts for the given tasks using suitable prompt engineering techniques.
- 2. Select and apply the appropriate prompting strategy (Zero-shot, One-shot, Few-shot, Chain-of-Thought, or Structured Prompting) based on the given scenario.
- 3. Include clear instructions, relevant context, necessary constraints, and the expected output format while designing each prompt.
- 4. Evaluate and refine the designed prompts to improve their clarity, effectiveness, and quality by applying prompt engineering best practices.
- 5. Design prompts for a variety of real-world applications, such as text generation, summarization, translation, question answering, code generation, and conversational AI.
- 6. Justify the designed prompt by briefly explaining the selected prompting technique and how it helps achieve the desired output.

Code of Conduct

*I _ **A.Sai Rohit (192472144)** _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **A.Sai Rohit**_____

SECTION A – **PROMPT ENGINEERING CHALLENGE**

Create a prompt to perform grammar correction on a given paragraph.

1. Aim

To design an effective AI prompt that performs **grammar correction** on a given paragraph by correcting grammatical errors, spelling mistakes, punctuation, capitalization, sentence structure, and word usage while preserving the original meaning and writing style.

2. Problem Statement

Many written paragraphs contain grammar, spelling, punctuation, and sentence structure errors that reduce readability and professionalism. The objective is to design a prompt that instructs an AI model to identify and correct these errors without changing the intended meaning of the text.

3. Selected Prompt Engineering Technique

Technique: Structured Prompting (Zero-Shot)

Reason for Selection

- The task is straightforward and does not require examples.
- Structured Prompting clearly defines:
 - Role
 - Task
 - Rules
 - Constraints
 - Output Format
- It improves consistency and accuracy in grammar correction.

4. Designed Prompt

You are an expert English grammar editor with strong knowledge of grammar, spelling, punctuation, capitalization, and sentence structure.

Your task is to correct the grammar of the given paragraph while preserving its original meaning and writing style.

Instructions:

1. Correct all grammatical errors.
2. Fix spelling mistakes.
3. Correct punctuation and capitalization.
4. Improve sentence structure only where necessary.
5. Do not change the meaning of the paragraph.

- 6. Do not add or remove important information.
- 7. Maintain the original tone and writing style.
- 8. Ensure the final paragraph is fluent, natural, and professional.

Input Paragraph:

[Paste the paragraph here]

Output Format:

Corrected Paragraph:

<Corrected text>

Corrections Made:

- Grammar:
- Spelling:
- Punctuation:
- Capitalization:
- Sentence Structure:

5. Prompt Components

Component	Description
Role	Expert English Grammar Editor
Context	Correct grammar while preserving meaning
Task	Grammar correction
Instructions	Grammar, spelling, punctuation, capitalization, sentence structure
Constraints	Do not change meaning or tone
Output Format	Corrected paragraph with list of corrections

6. Sample Input

Yesterday i goes to market and bought some fruits. The weather were very hot but i enjoys the trip because the peoples was friendly.

7. Expected Output

Yesterday, I went to the market and bought some fruits. The weather was very hot, but I enjoyed the trip because the people were friendly.

Corrections Made

Grammar

- goes → went
- buyed → bought
- were → was
- enjoys → enjoyed
- peoples → people
- was → were

Capitalization

- i → I

Punctuation

- Added comma after "Yesterday"
- Added comma before "but"

Sentence Structure

- Improved sentence flow while preserving the original meaning.

8. Prompt Engineering Strategy Used

Structured Prompting

The prompt is divided into multiple sections:

- AI Role
- Objective
- Detailed Instructions
- Constraints
- Expected Output Format

This structure helps the model understand the task more accurately and consistently.

9. Constraints Included

- Preserve the original meaning.
- Do not rewrite unnecessarily.
- Maintain the same tone.
- Correct only language-related mistakes.

- Produce grammatically correct English.
- Keep the paragraph natural and fluent.

10. Expected Output Format**Corrected Paragraph:**

<Corrected paragraph>

Corrections Made:

Grammar:

..

Spelling:

...

Punctuation:

...

Capitalization:

...

Sentence Structure:

...

11. Evaluation of the Prompt

Evaluation Criteria	Result
Clarity	Excellent
Specific Instructions	Excellent
Context Provided	Yes
Output Format Defined	Yes
Constraints Included	Yes
Preserves Meaning	Yes
Easy for AI to Follow	Yes

12. Prompt Refinement

Initial Prompt

Correct the grammar of the following paragraph.

Refined PromptProblems

- No role assigned.
- No context provided.
- No constraints.
- No output format.
- May rewrite the paragraph unnecessarily.

Refined Prompt

You are an expert English grammar editor.

Correct grammar, spelling, punctuation, capitalization, and sentence structure while preserving the original meaning and tone.

Do not add or remove information.

Output:

1. Corrected Paragraph
2. List of Corrections under Grammar, Spelling, Punctuation, Capitalization, and Sentence Structure.

Improvements After Refinement

- Assigned a clear AI role.
- Added detailed instructions.
- Included constraints.
- Specified the expected output format.
- Improved consistency and reliability of responses.

13. Real-World Applications

- Academic essay proofreading
- Student assignments
- Resume and CV editing
- Email correction
- Business reports
- Content writing
- Blog editing
- Research papers
- Professional documentation
- Social media content proofreading

14. Design Rationale (Justification)

The **Structured Prompting (Zero-Shot)** technique was selected because grammar correction is a well-defined task that does not require demonstration examples. By clearly specifying the AI's role, detailed instructions, constraints, and expected output format, the prompt ensures accurate correction of grammar,

spelling, punctuation, capitalization, and sentence structure while preserving the original meaning and tone. This structured design reduces ambiguity, improves consistency, and produces high-quality, professional results suitable for real-world writing tasks.

15. Conclusion

The designed prompt effectively guides an AI model to perform grammar correction with high accuracy. It includes a clear role, context, instructions, constraints, and a structured output format. After evaluation and refinement, the prompt becomes more reliable, consistent, and suitable for professional, academic, and everyday writing

Create a prompt that demonstrates **Function Calling** for retrieving weather information.

1. Aim

To design an effective prompt that enables an AI model to use **Function Calling** for retrieving real-time weather information from an external weather API based on the user's location or city.

2. Problem Statement

Users often ask questions such as:

- "What's the weather in Hyderabad?"
- "Will it rain tomorrow in Chennai?"
- "What is the temperature in Delhi?"

Since an AI model cannot always know real-time weather information, it should identify the required location and invoke an external weather function/API to fetch accurate weather details.

The objective is to design a prompt that demonstrates **Function Calling** for weather retrieval.

3. Selected Prompt Engineering Technique

Technique: Structured Prompting + Function Calling
Reason for Selection

- Function Calling enables the AI to interact with external APIs.
- Structured Prompting clearly defines the AI's role, instructions, constraints, and output format.
- It ensures reliable and accurate retrieval of live weather data.

4. Designed Prompt

You are an intelligent AI weather assistant capable of using Function Calling to retrieve live weather information.

Your task is to answer weather-related questions by calling the weather function whenever current weather information is required.

Instructions:

1. Identify the city or location mentioned by the user.
2. If the location is missing, politely ask the user to provide it.
3. Use the function:

`get_weather(city)`

4. Wait for the function response before generating the final answer.
5. Present the weather in a simple and user-friendly format.
6. Do not guess weather information.
7. Mention if the weather data is unavailable.

User Query:

[Enter user's weather question here]

Expected Output Format:

Location:

Temperature:

Weather Condition:

Humidity:

Wind Speed:

Advice:

5. Function Definition (Example)

Function Name:

`get_weather(city)`

Input:

```
{  
  "city": "Hyderabad"  
}
```

Output:

```
{  
  "temperature": "31°C",  
  "condition": "Cloudy",  
  "humidity": "72%",  
  "wind_speed": "14 km/h"  
}
```

6. Prompt Components

Component	Description
Role	AI Weather Assistant
Context	Retrieve live weather using Function Calling
Task	Call weather API and display results
Instructions	Detect city, invoke function, generate response
Constraints	Never guess weather information
Output Format	Structured weather report

7. Sample Input

What is the weather in Hyderabad today?

8. Function Calling Flow

Step 1 – User Request

What is the weather in Hyderabad today?

↓

Step 2 – AI Detects City

City = Hyderabad

↓

Step 3 – Function Call

get_weather("Hyderabad")

↓

Step 4 – Function Returns

```
{
  "temperature": "31°C",
  "condition": "Cloudy",
  "humidity": "72%",
  "wind_speed": "14 km/h"
}
```

↓

Step 5 – AI Response

Location: Hyderabad

Temperature: 31°C

Weather Condition: Cloudy

Humidity: 72%

Wind Speed: 14 km/h

Advice:
Carry an umbrella as the weather is cloudy and rain is possible. Stay hydrated.

9. Constraints Included

- Use Function Calling for live weather data.
- Do not generate fake weather information.
- Ask for the city if it is not specified.
- Display results in a structured format.
- Keep the response concise and user-friendly.
- Handle API errors gracefully.

10. Expected Output Format

Location:
Temperature:
Weather Condition:
Humidity:
Wind Speed:
Advice:

11. Evaluation of the Prompt

Evaluation Criteria	Result
Clear Instructions	Excellent
Function Calling Included	Yes
Role Defined	Yes
Context Provided	Yes
Constraints Included	Yes
Output Format Defined	Yes
Easy for AI to Follow	Yes

12. Prompt Refinement

Initial Prompt

Tell me the weather in a city.

Problems

- No role assigned.
- No Function Calling instruction.
- No output format.
- No handling of missing location.
- AI may guess weather information.

Refined Prompt

You are an AI Weather Assistant.

Whenever a user asks about current weather, identify the city and call the function:

`get_weather(city)`

If no city is provided, ask the user for the location.

Never guess weather information.

Display the response in the following format:

Location:

Temperature:

Weather Condition:

Humidity:

Wind Speed:

Advice:

Improvements After Refinement

- Defined AI role.
- Explicit Function Calling instruction.
- Added constraints to prevent hallucination.
- Included handling for missing location.
- Standardized the output format.

13. Real-World Applications

- Weather forecasting apps
- Smart assistants (Alexa, Siri, Google Assistant)
- Travel planning systems
- Agriculture advisory platforms
- Disaster management systems
- Flight and transportation services
- Tourism applications
- Mobile weather applications

14. Design Rationale (Justification)

The **Structured Prompting + Function Calling** technique was selected because weather information is dynamic and requires real-time retrieval from an external source. The structured prompt clearly defines the AI's role, instructions, constraints, and expected output, while Function Calling enables the AI to invoke a weather API instead of relying on stored knowledge. This approach ensures accurate, reliable, and up-to-date weather information, reduces hallucinations, and improves user trust.

15. Conclusion

The designed prompt effectively demonstrates how an AI assistant can use **Function Calling** to retrieve live weather information. By combining **Structured Prompting** with **Function Calling**, the AI can identify the user's location, invoke the appropriate weather function, and present accurate, real-time weather details in a clear and structured format.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technique Selection	30%	All key concepts (AI, ML, DL, GenAI, LLMs) correctly identified with clear hierarchical relationships	Most concepts correct; minor gaps in relationships	Several concepts missing or misclassified	Major conceptual errors; map incomplete
Output Effectiveness	25%	Real-world examples linked accurately to each concept	Examples mostly relevant	Few or generic examples	No real-world linkage shown
Prompt Craftsmanship	20%	Logical, well-organized structure with clear connectors	Mostly organized; minor structural issues	Some disorganization affecting clarity	Poorly structured, hard to follow
Prompt-Output Documentation	15%	Visually clear, creative, easy to interpret	Neat and readable	Cluttered or inconsistent formatting	Untidy, difficult to interpret
Design Rationale	10%	Concise labels/notes that add clarity	Adequate labeling	Minimal or unclear labeling	No supporting labels or explanation

Marks Evaluation:

Question No.	Technical / Concept(30%)	Application(25%)	Quality (20%)	Presentation (15%)	Documentation / Communication (10%)	Total Marks (100)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____